MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión C del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_hernandez_karen
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/hernandez_karen/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

Tu respuesta

La abstraccion es una manera de saber que cosas o caracteristicas de un objeto son necesarias para modelaralas ne un codigo en base a algun contexto, en el caso de un obejto como carro, podemos conservar cual es velocidad maxima,pero tal vez podemos descartar de que color es,ya que no es esencial para nuestro codigo.

Corrección. Correcta y completa: abstraer es elegir las características relevantes según el contexto. El ejemplo conserva la velocidad máxima y omite el color con justificación, como pide la pregunta.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

Tu respuesta

El constructor nos permite diferenciar los atributos de los parametros al momento de crear un nuevo objeto el constructor amolda los datos ingresados para darle la estructura que el tiene, se distingue de un metodo ya que no lleva tipo de dato al declararlo

Corrección. Bien que lo relaciones con la creación del objeto y que digas que no declara tipo de retorno. Pero el constructor no sirve para 'diferenciar atributos de parámetros' (eso lo hace this): sirve para inicializar el estado de la instancia. Además faltó decir que lleva el mismo nombre que la clase.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

Tu respuesta

```
Impide que su valor no pueda cambiar, el valor que se le asigna al inicio se mantendra durante toda la ejecucion
```

Corrección. Se entiende que final impide reasignar la variable, aunque la frase 'impide que su valor no pueda cambiar' dice lo contrario. No contestaste la segunda parte: con una referencia final a un arreglo no puedes cambiar la referencia, pero sí puedes modificar sus elementos.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

Tu respuesta

```
No basta ya que no compara el contenido de los String, compara las direcciones de memoria, para poder comparar el contenido debemeos de usar .equals()
```

Corrección. Correcta: == compara referencias (identidad, 'direcciones') y no el contenido; para comparar el texto se usa equals().

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

Tu respuesta

```
La diferencia es que en while primero evalua si la condicion se cumple para entrar en esa parte del codigo, si es falsa no entra, en el do-while se ejecuta aunque sea una vez esa parte del codigo, y al final se evalua la condicion, si es falsa,solamente se ejecuta una vez
```

Corrección. Correcta: while evalúa antes y puede hacer cero vueltas; do-while evalúa al final y ejecuta el cuerpo al menos una vez aunque la condición sea falsa desde el inicio.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{9, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla[0][i];
    System.out.println(i + ":" + total);
}
System.out.println((double) (total / 2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

Tu respuesta

```
0:9
4.0
P 4 2
```

Corrección. La línea 4.0 es correcta. 'P 4 2' tiene los valores correctos, pero la concatenación no agrega espacios: es P42. Escribiste '0:9' en vez de las tres líneas del ciclo 0:4, 1:6, 2:9; solo se reconoce el total final 9. Las otras dos líneas del ciclo no aparecen. Salida esperada: 0:4 / 1:6 / 2:9 / 4.0 / P42.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
i tomara el indice[2],ya que la condicion deberia de ser i<datos.length o i<=datos.length-1
```

Corrección. La clasificación es incorrecta: sí compila, pero truena en ejecución. La causa sí es correcta: con i <= datos.length se intenta leer datos[2], que está fuera del arreglo.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer piezas = 6;
System.out.println(piezas + 2);
System.out.println("P" + piezas);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Integer vuelve entero a piezas,P 8
```

Corrección. La clasificación 'Funciona' es correcta y se reconoce la idea de que Integer se usa como entero al sumar. La salida 'P 8' no es correcta: se imprimen dos líneas, 8 / P6, porque + 2 desempaqueta y suma, y con texto se concatena 6.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
static int medir(int valor) {
    if (valor > 0) return valor;
}
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
nos retorna el valor
```

Corrección. Incorrecta: no compila, porque el método int no retorna nada cuando valor es cero o negativo, y Java revisa todos los caminos. 'Nos retorna el valor' no da la causa.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

```
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
```

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
Public class Lampara{
    static final int LAMPARAS_MAXIMAS=10;
    ESTADO estado=apagado;

}

Lampara(Estado estado){
this.estado=estado;

}
```

Justificaciones

```
Maximo 10 lamparas se pueden crear
```

Corrección. Bien el atributo de instancia estado con el enum y valor inicial apagado (debía escribirse Estado estado = Estado.APAGADO), aunque no lo justificaste. La constante static final LAMPARAS_MAXIMAS está justificada, pero no corresponde a lo pedido: la constante natural era la etiqueta fija. Faltan la etiqueta (final String de instancia), el contador static int creadas, un constructor que reciba la etiqueta, asigne APAGADO e incremente el contador (el tuyo queda fuera de la clase y recibe el estado como parámetro), y los métodos encender() y la consulta.

R4 · Código con repetición

Pregunta

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

```
static int posicionMinima(int[] temperaturas)
```

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {7} → 0 · {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
{
int minimo=temperaturas[0];
int i;
for(i=1;i<temperaturas.length;i++){
    if(minimo>temperaturas[i]){
        minimo=temperaturas[i];
    }
}
return i;
}
```

Corrección. Bien el recorrido con for desde 1 hasta el último elemento y la comparación estricta, que conserva la primera aparición. Pero guardas el valor mínimo en lugar de su posición y al final devuelves i, que siempre vale temperaturas.length: {8,3,5} devuelve 3 y {7} devuelve 1. Había que guardar la posición (posicion = i) y devolverla. Falta la cabecera static int posicionMinima(int[] temperaturas).

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

```
static String mantenimiento(int ciclos)
```

Regla o valor recibido	Texto devuelto
Menor o igual que 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"
Cualquier otro	"NINGUNO"

Tu respuesta

```
{
    if(ciclos<=0){;
        System.out.println("INVALIDO")
    }
    else if(ciclos % 24 ==0){
        System.out.println("COMPLETO");
    }
    else if(ciclos %12 ==0){
        System.out.println("REVISION");
    }
    else if(ciclos % 4 == 0){
        System.out.println("BASICO");
    }
```

```
    else{
        System.out.println("NINGUNO");
    }

}
```

Corrección. La selección es correcta: primero se descartan cero y negativos, y luego se revisa 24, 12 y 4 en el orden adecuado, con else final. Pero el método debía devolver el texto con return y no imprimirlo con System.out.println, que además el enunciado prohíbe. Faltan la cabecera static String mantenimiento(int ciclos) y un punto y coma después de println("INVALIDO").

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.